

re_mqtt_proc (Mqtt_Json_Format_V3.0_Dual_Broker) - Deep review

Branch autofix/mqtt-high-2026-10-03, commit e670bb3 - 03 Oct 2026

1. Verdict

Not ready for release. The branch fixes 6 of the 49 findings from the 03-Oct review fully and 7 partly. This deeper pass found **16 High findings** (15 new, plus R4, which extends F7), 26 Medium and 19 Low. Five are worth knowing before anything else:

- Two root command-injection paths are still open.** One is IPSEC fields loaded with eval (N1, still open). The other is new (R2): IPSEC subnet fields and the DLMS Ethernet meter IP flow into a shell command that dcu_mon_proc runs.
- MQTT set_cfg can never work** (29-Sep #1, still open). It looks up a Redis field mqtt1 that nothing writes, so every request fails without writing anything.
- The server is sent wrong data:**
 - CT and PT ratio values are swapped (R11).
 - Billing records are keyed with a month label instead of the billing date (R10).
 - Event values shift against their OBIS codes (R12).
 - FetchDay replies are not matched to their request (R3).
 - ACKs carry a fixed SEQ_NUM instead of the request's (R5).
- The daemon crashes** on 5 command types when the DCU serial is missing from Redis or Redis is down (R1).
- The bench failures now have causes in the code:**

Bench result	Cause
Health message invalid JSON	R7
40 health slots / duplicate ID 25	R8
Billing date labels malformed	R10
EVENT data empty	R13
Genus PT ratio 0	R11
get_cfg mqtt1 key	29-Sep #1

Method: five reviewers each read every line of one part of the module. The parts were:

- mqtt_connect.c, in two halves
- main.c with the logging and health code
- the file generators
- set_cfg/get_cfg/Modbus

Each reported findings with a traced failure path. I then re-checked every High against the source myself. Findings that could not be traced are marked **Plausible**; all others are **Confirmed**. Nothing was run on the DCU.

2. The branch's own fixes, reviewed critically

Fix	Holds?	Problem found in the fix
F1 MODEM whitelist	Yes	A credential starting with # becomes a pppd comment, so authentication fails silently (R-L14). ppp_monitor.sh still generates shell code.
F2 file names	Yes	Generator failures are still not reported to the server.
F3, F4, F9, F8	Yes	None.
F10 js_str	Crash gone	Numeric JSON values now become "" silently (PT/CT ratio, port, met_id), see R-M15.
F5 logging lock	Race gone	The lock is held across Redis and network-log I/O (up to 600 ms per line), which stalls Paho threads (R-M9). The throttles use the wall clock.

F7 timeout	Partly	The timer stops while the FetchDay broker is down , so a stale SEQ_NUM can stay on cyclic data for as long as the outage lasts (R-M1). No failure reply is sent on timeout. A late poller reply after the DEL answers the next FetchDay (R3). fetchday_reset_state() runs only on timeout, not on the other failure paths (R-M2).
-------------------	--------	---

3. Status of the 49 findings from 03-Oct (plus N1, 29-Sep)

Status	Findings
Fixed (6)	F1, F3, F4 (callee), F8, F9, F10 (crash)
Partly (7)	F2, F5, F7, F14, F27, L7, L17
Open (36)	F6, F11, F12, F13, F15, F16, F17, F18, F19, F20, F21, F22, F23, F24, F25, F26, F28, L1-L6, L8-L16, L18-L21
Also open	N1 (IPsec eval injection), 29-Sep #1/#2/#3 (MQTT set_cfg/get_cfg field names), 29-Sep #11 (cyclic Modbus {})

Severity changes:

Finding	Change	Why
F15	→ High	The Redis reply leaked in read_meter_status is about 20-25 MB/day at 20 meters and 5-minute cycles, enough to exhaust memory within weeks.
F21	→ High	Traced: a serial meter without a status entry is given an Ethernet meter's COMM_STATUS.
L3	→ Medium	The restart names with spaces never match dcu_mon, so MODBUS_RTU, DLMS_SERIAL and DLMS_ETHERNET set_cfg take effect only after a reboot.

4. New findings - High (16)

ID	Location	Problem	Failure scenario	Fix	Ver.
R1	mqtt_connect.c:3864, 3946, 3987, 4023, 4096	strcmp(cmd.args[0], dcu_sn) with dcu_sn NULL. The branch fixed only FetchDay.	dcu_info.serial_num missing, or Redis down (no reconnect, L1), then any GetDay, ReadModbus, get_cfg, set_cfg or Reset causes SIGSEGV, and it repeats after every restart.	One helper that checks NULL, compares and frees; use it in all 7 handlers.	C
R2	set_cfg_request.c:412-415, 1199-1225 → dcu_mon_proc.c:2553, 2583	IPSEC RIGHT_SUBNET/RIGHT_SUBNET_MASK and DLMS_ETHERNET IP_ADDR are stored unchecked. dcu_mon_proc pastes them into system("eth_nat.sh start %s %s %s") as root.	set_cfg IPSEC "RIGHT_SUBNET": "10.0.0.0;reboot", then START_TRANS_MODE on an Ethernet meter, runs reboot as root. This is independent of N1.	Accept only dotted-quad IPv4 (inet_pton) in set_cfg. In dcu_mon, validate again and use execv, not a shell.	C
R3	mqtt_connect.c:2629-2742; main.c:1011-1031	OD replies are popped from mqtt_command_resp without checking seq_no. A reply that arrives after the timeout's DEL stays in the list.	FetchDay A (meter X) times out and the poller answers late. The next FetchDay B (meter Y) gets X's data under B's SEQ_NUM, and B's own reply answers C. The offset continues for every later request.	Keep the pending request in a struct and drop replies whose seq_no does not match. Reply "busy" while a request is pending. Clear the list before queuing.	C
R4	mqtt_connect.c:3768, 2938-	cpy_cmd = cmd runs for every	A periodic get_cfg during a 7-day	Do not overwrite. Pass the	C

	2945; file_gen_main.c:905-908	command. OD replies take their SEQ_NUM from it, and OD EVENT takes its dates from it.	FetchDay LS gives days 3-7 get_cfg's SEQ_NUM. A FetchDay EVENT followed by any other command makes the query LIKE '%', so the server gets all events (cap 1000).	transaction and dates explicitly to the generators.	
R5	cmd_resp.c:229; mqtt_connect.c:3822-3853	The ACK SEQ_NUM is a constant per type (1001, 1002, 3001, 2102, 2004, 1003, 1009). Spec 5.3 says it must echo the request. The ACK is sent after meter validation, but spec 5.1 says “ACK first”.	The server cannot correlate ACKs. If it enforces ACKs it resends, and commands run twice.	ack_msg_reply(const char *seq) with cmd.transaction, sent before validation.	C
R6	mqtt_connect.c:3452, 3529-3599	GetDay has no cap on the date range. The per-day loop runs on the worker and publishes even empty days.	GetDay LS over a year, or 1902→2037, blocks cyclic data, heartbeat and commands for minutes to hours. dcu_mon may then restart the daemon mid-way.	Strict date check plus a maximum span (spec limit), with a failure reply. Process one day per loop iteration.	C
R7	health_status.c:993-1049; producer modem_monitor.sh:531-546	Cause of the invalid health JSON: every Redis string is written with "%s" and no escaping, and the modem script stores the raw AT replies (CR/LF in IMEI and ISP).	Every health message on a DCU with a modem fails to parse. A " in a meter name breaks it too.	Build it with cJSON (or a json_escape()). Also trim CR/LF in the script.	C
R8	health_status.c:284-354, 1032-1117; www/redis_worker.py:1774, 1813	Cause of 40 slots / 27 empty / duplicate ID: the web UI always writes num_meters as the capacity (30+5+5), and health lists every slot. ID is the DLMS address, which is not unique.	The server shows phantom meters, and statuses with a duplicate ID map to the wrong meter.	Skip disabled or empty slots. Agree a unique ID (serial, or port+slot) with the server.	C
R9	general.h:142; main.c:25, 182-185	MAX_METERS 20, but the product supports 40 DLMS meters. load_active_meters stops at 20.	Meters 21 and up get no cyclic data, and GetDay/FetchDay for them returns “Invalid meter name”. Which 20 survive changes with Redis order.	Size from the feature limits (≥ 40) or dynamically. Validate command meters against the configuration.	C
R10	dlms_cdf_billing.c:148-164	Cause of the malformed billing dates: “.000” is appended to the month label column (bill_date = “Oct 2026”). The real billing date 0_0_0_1_2_255 is discarded.	Every billing record is keyed “Oct 2026.000” or “curr mon 2026.000”.	Use 0_0_0_1_2_255 (normalised) as the key.	C
R11	file_gen_main.c:1205-1213	CT and PT ratio swapped: OBIS_CT_RATIO gets pt_ratio and OBIS_VT_RATIO gets ct_ratio.	The server stores PT as CT and CT as PT. This, plus R-M15, is the likely cause of “Genus PT ratio 0”.	Swap the arguments. Confirm the poller's field meaning first.	C
R12	dlms_cdf_event.c:328-334, 515-539, 573-592	Values equal to FFFF are dropped per record, while PARAMS comes from record 0.	Values after a dropped column shift onto the next OBIS code, so the server stores, for example, a current under a voltage OBIS.	One value per column (" for FFFF). PARAMS from the column list.	C
R13	file_gen_main.c:2216; dlms_cdf_event.c:144-169, 385-391; mqtt_connect.c:2551, 2940-2977	Causes of empty EVENT data: cyclic covers today only. FetchDay asks the poller for “all”, then filters by the request dates and drops the rest. “No data” is reported as success. Category 6 becomes “all”. event_category can be used	The server rarely sees events, and cannot tell “none” from “failed”.	Cyclic: events newer than the last sent. FetchDay: send what was fetched. Distinct no-data status. Map category 6 and initialise the variable.	C

		uninitialised.			
R14	dlms_cdf_billing.c:901-904, 196-200	The end-month test ignores the year, so the OD table is dropped part-way through a multi-year range.	FetchDay BILLING Sep 2025 to Oct 2026 drops the table after Oct 2025. The remaining months fail and the reply still says success.	Compare year and month, and drop once after the loop.	C
R15	dlms_cdf_mn.c:329-340, 597-604	MN PARAMS come from day 1 even when day 1 has no data.	GetDay MIDNIGHT where the first day is missing gives "PARAMS": [] with populated records, so the response is unusable.	PARAMS from the first non-empty day, or from the column names.	C
R16	set_cfg_request.c:955, 1034	Modbus RESP_TIMEOUT is stored raw (seconds per spec and web UI), but the masters read milliseconds.	RESP_TIMEOUT 11 becomes 11 ms: every poll times out and the data stops. set_cfg still replied SUCCESS.	Multiply by 1000 with a range check (and divide in get_cfg), or change the spec to ms.	C

R4 is the part of F7 still open, plus a new EVENT impact.

5. New findings - Medium (26)

ID	Location	Issue	Ver.
R-M1	main.c:1011-1016	F7 timeout is suspended while the FetchDay broker is down, so the stale SEQ_NUM stays on the other broker's cyclic data (branch fix flaw)	C
R-M2	mqtt_connect.c:2695, 2750, 3040, 3905	LS day counters left over on 3 paths; a second FetchDay is accepted while one is pending; Fetchday_cmd_broker is set before the serial check	C
R-M3	mqtt_connect.c:3946-4101, 2452	Parsed command tree and dcu_sn leaked on most paths. A wrong-serial command leaks without authentication, so any client that can publish can grow memory	C
R-M4	mqtt_connect.c:943-966	Publish timers reset on every reconnect, so a link flapping faster than the interval never sends meter data	C
R-M5	mqtt_connect.c:1588; main.c:1394	clean_session defaults to 0 when missing: queued commands replay on reconnect; a command sent to both brokers runs twice	P
R-M6	mqtt_connect.c:3666-3699 → dlms_cdf_event.c:178-225	SQL injection via GetDay EVENT category args[4] (read-only; FetchDay whitelists, GetDay does not)	C
R-M7	mqtt_connect.c:2459-2489	calculate_num_days accepts invalid or trailing-garbage dates; raw strings go to the poller while the DCU uses normalised ones	C
R-M8	health_status.c:186-214, 987-988	Health vs spec v0.20: fixed SEQ_NUM "0001"; ACTIVE_SINCE is a duration not a timestamp; TIME_SYNCED is not YES/NO; fields missing	C
R-M9	dbg_log.c:359-409; main.c:992, 1657	Log lock held across Redis/netlog I/O stalls Paho threads; dcu_netlog_poll runs on 2 threads unlocked (branch fix side effect)	P
R-M10	main.c:1003, 1196; dbg_log.c:266	About 25 MB of log per day: only 3-4 h of history kept; logging on by default; flash wear	C
R-M11	general.h:16 vs health_status.c:18	Two hiredis header trees can mix in one binary	P

		(redisReply ABI differs between 0.x and 1.x)	
R-M12	main.c:1382-1438	MQTT config read once at start; a Redis error then disables both brokers forever while the heartbeat stays green	P
R-M13	main.c:1081-1090	Cyclic profile is wall-clock “today”, so the blocks after the last cycle before midnight are never sent cyclically	C
R-M14	dlms_cdf_* readers	No sqlite3_busy_timeout; BUSY treated as end of data, so partial profiles go out as success and the OD table is then dropped	P
R-M15	file_gen_main.c:72-79	js_str (branch F10) turns numeric values into "" (port, PT/CT ratio, met_id)	P
R-M16	file_gen_main.c:1601, 913	Every cyclic meter-data message has SEQ_NUM “0003”; the counter is not 16-bit	C
R-M17	file_gen_main.c:916, 1602; OD wrappers	DATATYPE vs spec DATA_TYPE; LS has no DATE; OD replies use a different layout from the cyclic message	C
R-M18	ls.c:329; mn.c:123; billing.c:144; event.c:310	NULL DB values sent as “0.0”, indistinguishable from a real zero	C
R-M19	ls.c:694; mn.c:682; billing.c:1044; event.c:772	strcpy of a 512-byte path into the caller’s 128-byte buffer (stack overflow with a long base path plus EVENT category)	P
R-M20	dlms_cdf_event.c:212-248	1000-event cap keeps the oldest and drops the newest; about 12.8 MB allocation	C
R-M21	general.h:318-320; modbus_json_export.c:27-30	Modbus device/register limits (2/10/64) differ from the masters (32/128), so devices beyond them are invisible to set_cfg, get_cfg and ReadModbus	C
R-M22	mqtt_connect.c:3774; set_cfg_request.c:175	Every set_cfg value, passwords and PSK included, is logged and pushed to the diag list the web UI reads	C
R-M23	set_cfg_request.c (all handlers)	set_cfg replies SUCCESS when values were rejected, ignored, half-applied or nothing was written	C
R-M24	mqtt_cmd_validation/	Validator never compiled or called; even wired in, it only length-checks the injection fields and rejects spec messages	C
R-M25	cmd_resp.c:136-197	command_reply does not echo METER/DATE keys (spec 4.3.2)	C
R-M26	mqtt_connect.c:2936	OD EVENT event_category[32] used uninitialised when the poller omits event_type	C

6. New findings - Low (19)

ID	Location	Issue
R-L1	dbg_log.c:46-56, 140-152	Diag throttles use time(), so they stall after a backward clock step; a NIL diag_enable keeps diag on
R-L2	mqtt_connect.c:4734-4790	Each publish stamps last_message_time on both brokers, which hides a failing broker in the web UI
R-L3	general.h:315	SEQ_NUM silently cut to 15 characters; written unescaped into OD JSON

R-L4	main.c:152-155	load_active_meters leaks the reply on a Redis error, every 3 s
R-L5	get_cfg_request.c, modbus_json_export.c, main.c, cmd_resp.c	Implicit declarations of pointer-returning functions; no -Werror (the same class of bug as F4)
R-L6	Makefile	Hard-coded user paths; objects written into another user’s tree; no -g, stack protector or FORTIFY
R-L7	dbg_log.c:179-203	A failed reopen after rotation stops file logging until restart; perror per line when the cfg file is missing
R-L8	main.c:917-918	32-bit time_t overflow in PUB_DUE with an absurd interval, so it publishes every loop
R-L9	main.c:1632-1648	dcu_netlog_send used before dcu_netlog_init (P)
R-L10	general.h	No include guard; MAX_METERS 20 vs 64; static prototype in a header
R-L11	file_gen_main.c:1156-1253	NAMEPLATE carries OBIS 00_00_19_01_00_ff twice; the IP comes from a removed key
R-L12	ls.c:373; mn.c:172; event.c:376	DROP TABLE is decided by the substring “od_” in the table name
R-L13	event.c:176-203; ls.c:76-79	SQL built with snprintf; multi-statement sqlite3_exec for DROP
R-L14	set_cfg_request.c:281-287	F1: a leading # in a credential becomes a pppd comment
R-L15	get_cfg_request.c:1029, 1042	MODBUS_TCP float parameters truncated to integers
R-L16	modbus_json_export.c:1083-1122; mqtt_connect.c:2389	ReadModbus “success” with an empty result; a string SLAVE_ID matches the wrong device
R-L17	set_cfg_request.c:1117-1204	DLMS set_cfg truncates over-long values instead of rejecting them
R-L18	file_gen_main.c:1322, 242 etc.	INST file uses a relative path; full JSON printed to stdout every cycle
R-L19	file_gen_main.c:1803, 1838	LOG_ERROR(stderr, ...) passes a FILE* as the format (dead code path)

7. Suggested order of work

1. **Security, now:**
 - R2 and N1: IPv4/charset validation in set_cfg, and `execv` instead of `system/eval` in `dcu_mon` and `ipsec_monitor`.
 - R-M22: stop logging secrets.
 - F13: mask secrets in `get_cfg`.
2. **Crashes and memory:** R1, F15, R-M3, R-M19.
3. **Wrong data to the server:** R10, R11, R12, R13, R14, R15, R16, R7, R8, R9, R-M18.
4. **Protocol correctness:**
 - R5 (ACK echo) and R-M16 (SEQ_NUM counter).
 - R3, R4, R-M1, R-M2: one pending-request struct with an explicit transaction. This also removes F6/F7.
 - 29-Sep #1/#2/#3 (MQTT set_cfg field names).
5. **Robustness:** R6, R-M7 (strict dates, span limits), R-M14 (SQLite busy), L1 (Redis reconnect), R-M12.
6. **Build hygiene:** prototypes, `-Werror=implicit-function-declaration` `-Werror=int-conversion`, one hiredis, vendored netlog/heartbeat. The compiler would then catch the R-L5 class of bug.

8. Cross-check against earlier reviews and bench results (added after first issue)

The reviewers compared their findings against the 03-Oct review only. A check against the 29-Sep and 02-Oct MQTT reviews, the 30-Sep and 03-Oct `dcu_mon_proc` reviews, and the 03-Oct bench results shows that some “new” findings had **already been reported**.

Already reported:

Deep-review ID	Reported as
R1	02-Oct N16
R2	02-Oct N1, and <code>dcu_mon_proc</code> 03-Oct N1

R4	F7 and 02-Oct N9
R6	02-Oct N5
R12	02-Oct N4
R14	02-Oct N3
R15	02-Oct N13
R13 (category 6 part)	02-Oct N8
R-M2	02-Oct N6
R-M3	02-Oct N16
R-M6	02-Oct N2
R-M19	02-Oct N20
R-M20	02-Oct N21
R-M23	F20 and 02-Oct N14
R-M26	02-Oct N7
R-L2	29-Sep #7
R-L6	03-Oct dcu_mon_proc review
R-L13	02-Oct N2

Symptom reported by the bench test, cause new in this review:

ID	Bench symptom
R5	ACK SEQ_NUM not echoed
R7	Health message invalid JSON (F17 already named the unescaped code; the CR/LF from modem_monitor.sh is new)
R8	40 health slots / duplicate ID
R10	Billing date labels malformed
R11	Genus PT ratio 0
R13	EVENT data empty (cyclic today-only, FetchDay re-filter, no-data reported as success)

Genuinely new:

Severity	Findings
High	R3, R9, R16; plus the root causes in the table above
Medium	R-M1, R-M4, R-M5, R-M7, R-M8, R-M10, R-M11, R-M12, R-M13, R-M14, R-M15, R-M16, R-M17, R-M18, R-M21 (limit mismatch), R-M22 (argument logging and diag list), R-M24, R-M25
Low	R-L1, R-L3, R-L4, R-L7, R-L8, R-L9, R-L10, R-L11, R-L12, R-L14, R-L15, R-L16, R-L17, R-L18, R-L19
Severity raised	F15 → High, F21 → High, L3 → Medium

Full per-area working notes with code quotes are in *review_notes_2026-10-03/* next to this file.